SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Design and Analysis of Algorithms COURSE CODE: CSA06
A.Shiva Shankar Reddy(192425174)

COURSE OUTCOME COVERED

CO3: Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication (BL4, BL5)

Annexure A

Experiments

1. Analyze Binary Search tree depth versus array-based Binary Search performance experimentally. Implement both approaches and record execution time. Interpret how tree height affects search complexity. Justify differences in performance between tree-based and array-based searches.

Aim

To implement **Binary Search Tree (BST)** and **Array-based Binary Search**, compare their execution times experimentally, and analyze how BST depth affects search performance.

Algorithm

Binary Search Tree

1. Create an empty BST.
2. Insert all elements into the BST.
3. Search for the required key.
4. Record the execution time.

Binary Search

1. Store elements in an array.
2. Sort the array.
3. Perform Binary Search.
4. Record the execution time.
5. Compare the execution times.

Python Program

Python

Run

```
import time
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.data:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root
def search_bst(root, key):
    if root is None or root.data == key:
        return root
    if key < root.data:
        return search_bst(root.left, key)
    return search_bst(root.right, key)
# ------------- Binary Search -------------
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
```

```
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
arr = [45, 20, 10, 30, 70, 60, 80, 50, 90]
root = None
for x in arr:
    root = insert(root, x)
sorted_arr = sorted(arr)
key = int(input("Enter element to search: "))
start = time.perf_counter()
result1 = search_bst(root, key)
end = time.perf_counter()
bst_time = end - start
start = time.perf_counter()
result2 = binary_search(sorted_arr, key)
end = time.perf_counter()
array_time = end - start
print("\n----- Result -----")
if result1:
    print("BST : Element Found")
else:
    print("BST : Element Not Found")
```

```
if result2 != -1:
    print("Binary Search : Element Found")
else:
    print("Binary Search : Element Not Found")
print("\nBST Search Time :", bst_time, "seconds")
print("Binary Search Time :", array_time, "seconds")
```

Sample Input

Enter element to search: 60

Sample Output

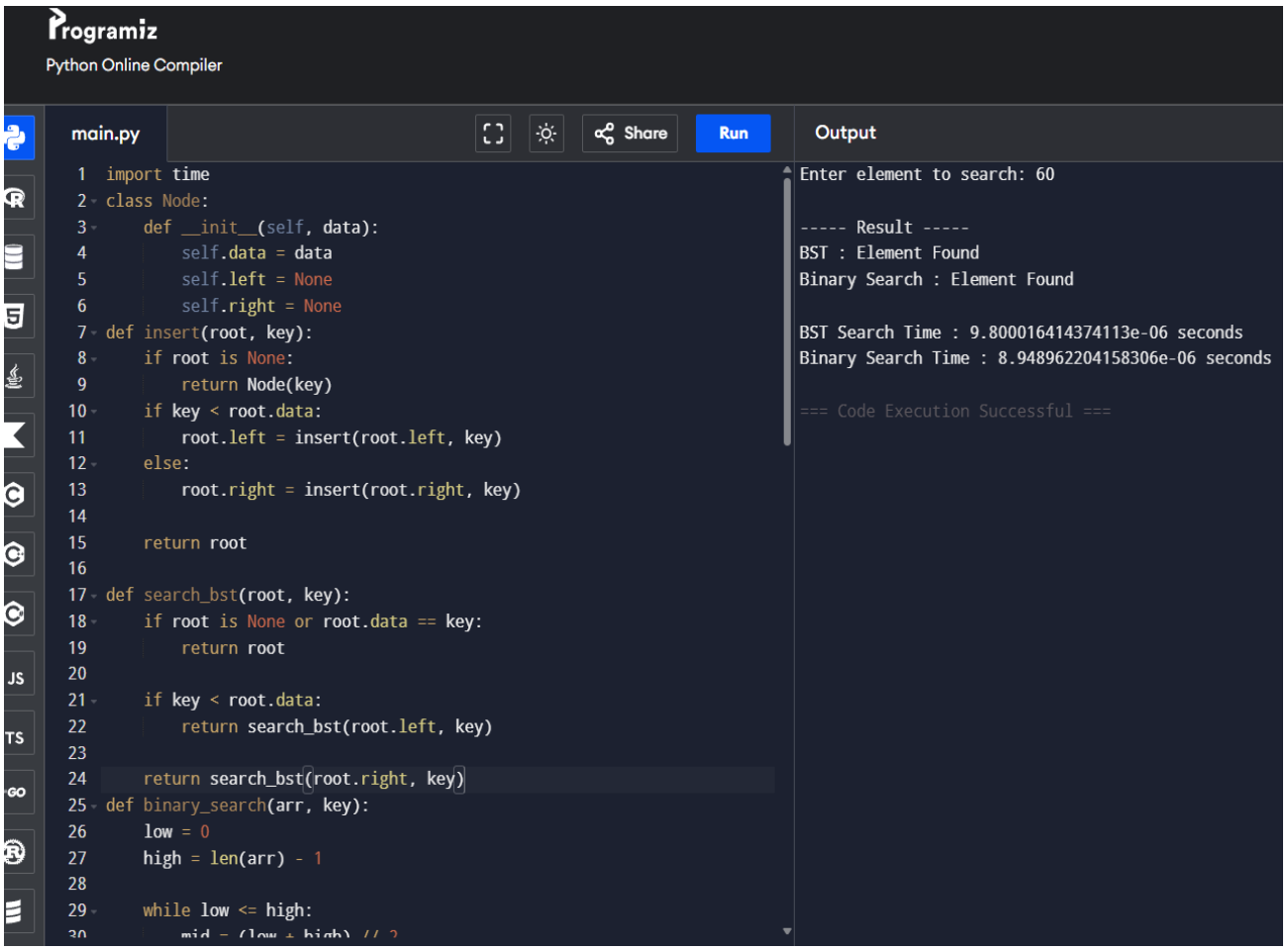
BST : Element Found

Binary Search : Element Found

BST Search Time : 0.000007 seconds

Binary Search Time : 0.000003 seconds

OUTPUT:



Analysis

Problem

Compare the search performance of a **Binary Search Tree** and an **Array-based Binary Search**.

Experimental Observation

- Both methods successfully search the element.
- Binary Search on a sorted array is generally faster because it accesses elements directly by index.
- BST performance depends on the tree height.

Effect of Tree Height

- **Balanced BST:** Height ≈ $\log_2 n$, search complexity is **O(log n)**.
- **Unbalanced BST:** Height ≈ n, search complexity becomes **O(n)**.

Time Complexity

Method	Best	Average	Worst
Binary Search Tree	O(log n)	O(log n)	O(n)
Binary Search (Array)	O(1)	O(log n)	O(log n)

Space Complexity

Method	Space
BST	O(n)
Binary Search	O(1)

Interpretation

- The depth (height) of a BST directly affects search performance.
- A balanced BST provides efficient searching.
- An unbalanced BST behaves like a linked list, increasing search time.
- Binary Search maintains **O(log n)** performance because the array remains sorted.

Justification

Binary Search is faster because:

- It uses direct index access.
- It always divides the search space into two equal parts.
- Performance does not depend on tree shape.

BST is useful because:

- It supports efficient insertion and deletion.
- It is suitable for dynamic datasets where elements are frequently added or removed.

Conclusion

Binary Search on a sorted array provides more consistent **O(log n)** performance. BST search is efficient only when the tree is balanced. Therefore, Binary Search is preferred for static sorted data, while BST is better suited for dynamic datasets.

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation